

Better AI accuracy and performance

Aleksey Savateyev

Cloud and AI Platforms

Microsoft

Data use cases in AI



Conversational AI

Conversational context

Model result optimizations

Auditing



Semantic Caching

Drastically reduces latency

Saves on token consumption

Reduces latency



AI Agents

Agent Memory

Agent-to-agent transactions

Statefulness

Logging



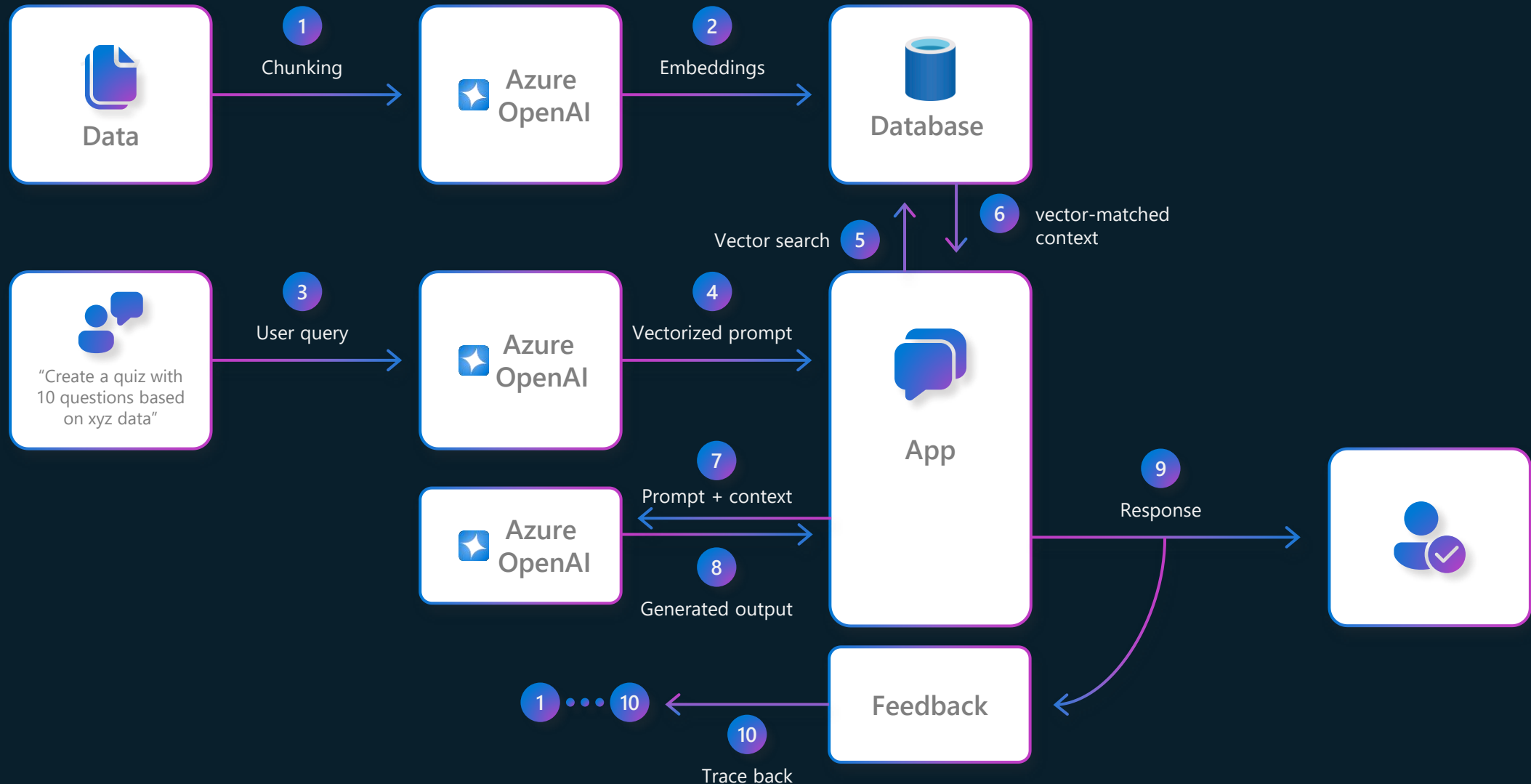
Enterprise Knowledge Graph

Breaks the data silos

Connects/unifies loosely connected data

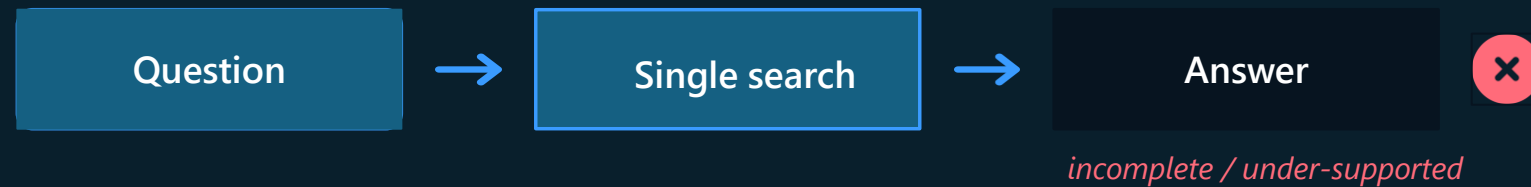
Remembers the complex insights

Typical Retrieval Augmented Generation (RAG) Architecture



The problem: Single-pass RAG breaks on complex questions

Many real questions require multi-hop reasoning across many data items. A single search-and-answer pass can't get there.



Evidence is scattered

One query can't surface everything across a large, heterogeneous corpus spanning many documents and modalities



Answers fall short

A single pass leaves gaps. Missing or under-supported evidence is never noticed, let alone filled



The stakes are high

In specialized or regulated domains (legal, medical, financial, etc.) answers demand completeness and accuracy

How to improve RAG

- Semantic reranking on top of vector search results
 - Typically produces marginally better results
 - Most context is already ignored by the time reranking starts
- Hybrid search - combining semantic search with full-text search with subsequent re-ranking
 - Better semantically relevant results
 - Still cannot answer nuanced/multi-hop queries, still misses context
- Better: use a combination of the above and more - “understand” the full context and/or perform diverse search iteratively
 - a) Create a connected model (graph) of structured portion of data corpus
 - b) Employ multi-stage diverse search with AI-assisted query rewriting on unstructured

Semantic Reranker in Azure Cosmos DB

Improve search and query relevance with AI-powered ranking

- **More relevant results** – Reorders results based on meaning and context, not just keyword matches
- **Works with existing searches** – Enhances vector, full-text, and hybrid search workloads
- **Better AI application quality** – Improves grounding for RAG, copilots, agents, and question-answering systems
- **No complex ranking pipelines** – Eliminates the need for custom scoring and manual relevance tuning
- **Integrated experience** – Add semantic relevance directly within Azure Cosmos DB

Public Preview

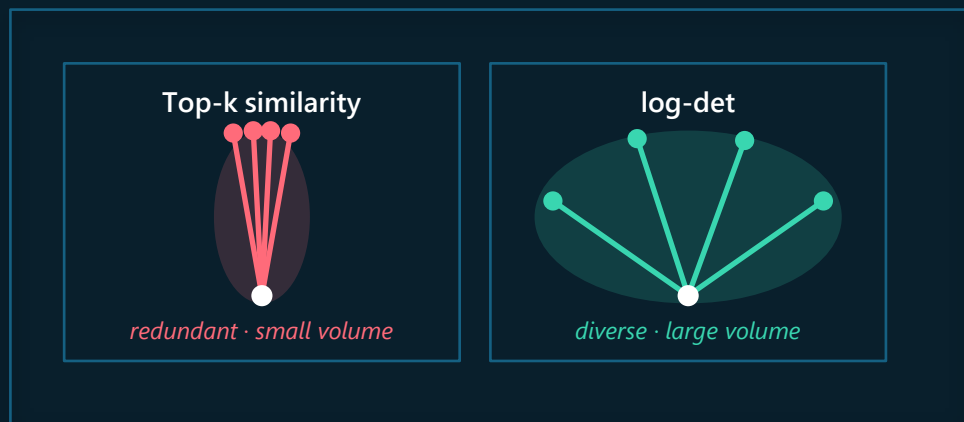
DiverseRAG: diversity-based agentic retrieval



Steps 2–6 repeat until knowledge gaps are filled, then the final answer is synthesized.

Diversity selection via maximizing log-determinant

The idea



Max $\log \det(\mathbf{G})$, $\mathbf{G} = \mathbf{V}\mathbf{V}^T$, is the volume spanned by the chosen vectors. Near-duplicate candidate vectors collapse it toward $-\infty$; diverse candidate vectors maximize it.

Tuning parameters



rescale_power

- Scale each vector by (cosine similarity to the query) ^ power before selecting.
steers diversity inside the relevant region instead of rewarding off-topic outliers.



eta (η)

- **0** → pure Gram-Schmidt.
- **>0** → regularized $\log \det(\mathbf{I} + (1/\eta) \cdot \mathbf{V}\mathbf{V}^T)$, numerically stabler when vectors are near-dependent.

hybrid search



diversity (log-det) prune



optional rerank



context to LLM

Diversity-based RAG concepts

APPROACH	OPTIMIZATION TARGET	WHAT IT FAVORS	TYPICAL EFFECT ON RAG GROUNDING
Top-k (baseline)	Maximize $\sum \text{similarity}(\text{query}, v_i)$	High individual relevance	Can be redundant; may miss complementary evidence for broad queries
DivDet (set selection)	Maximize diversity matrix determinant	Relevant + non-overlapping coverage	Improves coverage of facets; reduces repeated evidence
DiverseRAG (two-stage)	Apply Top-k/DivDet twice with LLM query augmentation	Better query targeting + diverse evidence	Boosts comprehensiveness/diversity while staying retrieval-efficient

Agentic Retrieval Toolkit for Cosmos DB implements DiverseRAG

Why it works



Iterative retrieval + reasoning

Drafts an answer, finds what's missing, then retrieves again, across rounds.



Hybrid retrieval

Combines vector search, full-text search, diversity selection, and optional semantic reranking.



Gap-aware decomposition

Breaks each gap into focused sub-questions and gathers evidence for every one.

Where it's useful



Complex, multi-topic questions

Span many documents and modalities at once.



High-stakes applications

Legal, medical, financial, and real-estate.



Large, heterogeneous corpora

Where one query can't surface all the relevant facts.



Enterprise knowledge bases

Structured + unstructured data across many collections.

Basic RAG vs GraphRAG vs DiverseRAG

	No Augmentation	Basic RAG	GraphRAG	DiverseRAG
Context Awareness	None	Some	Deep	Deep
“Direct hit” search results	✗	✓	✓	✓
Topically relevant connections	✗	✗	✓	partial
Deep and sparse connections	✗	✗	✓	partial
“Question behind the question”	✗	✗	partial	partial
Hallucination Risk	High	Low	Very Low	Very Low
Supports Hallucination Checking	✗	✓	✓	✓
Provides source references	✗	✓	✓	✓
Answers direct questions	✗	Usually	✓	✓
Answers nuanced questions	✗	✗	✓	partial
Explores adjacent topics and multiple perspectives	✗	✗	partial	partial
Reasons across many documents	✗	~10	✓	✓
Time to response	very fast (~5s)	fast (~5-10s)	medium (~10-30s)	fast (~5-10s)

Agent Memory Toolkit for Azure Cosmos DB

Build scalable memory systems for AI agents faster

- **Simplified agent memory architecture** – Store, update, and retrieve memories through a simple developer interface
- **Accelerate development** – Eliminate the need to build custom memory pipelines from scratch
- **Improved retrieval relevance** – Built-in memory processing and retrieval orchestration
- **Scalable and durable foundation** – Designed for production-ready agent and copilot applications
- **Focus on application logic** – Reduce infrastructure complexity and service integration effort

Public Preview



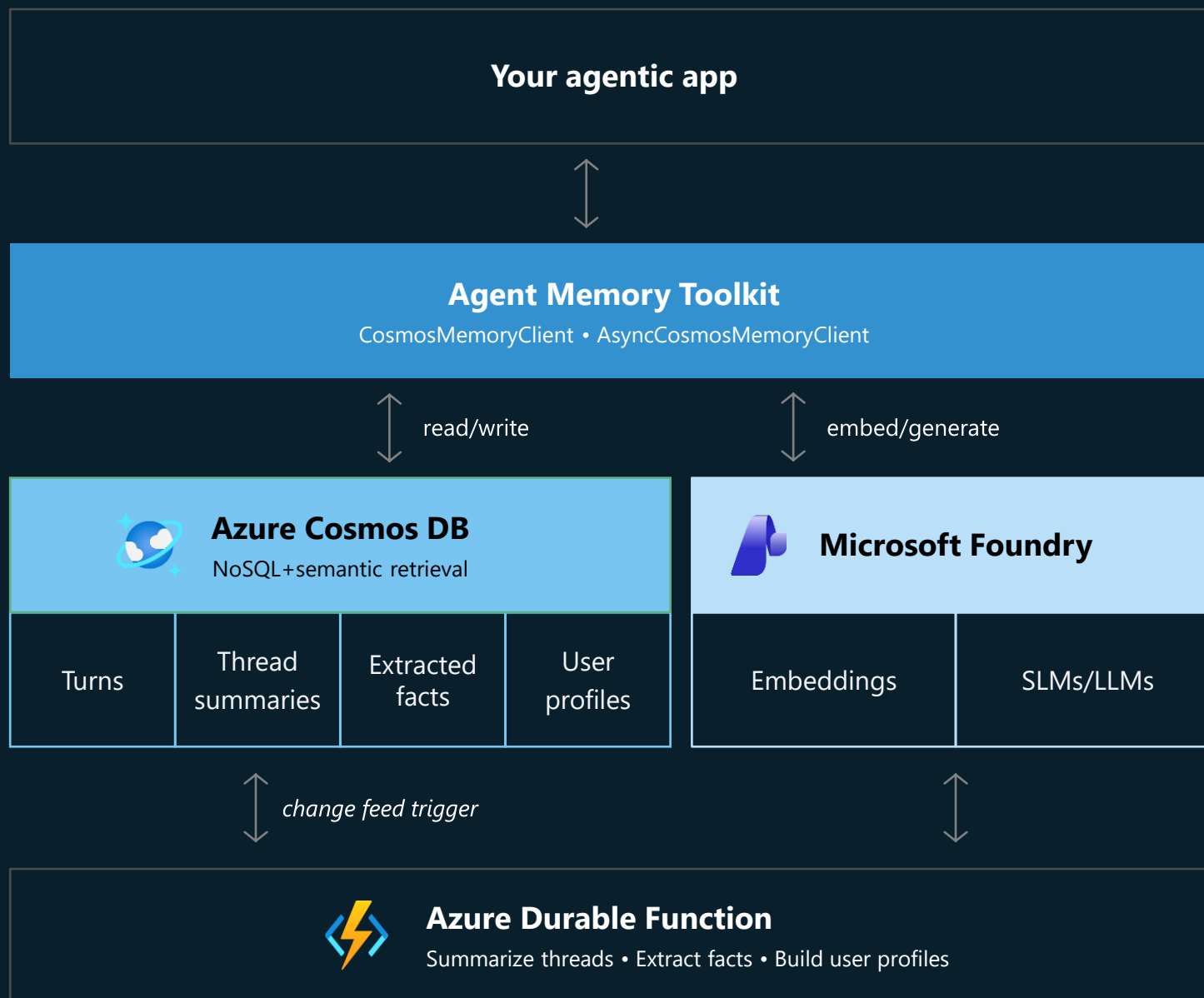
Azure Cosmos DB

Agent Memory Toolkit

A Python SDK & serverless
architecture

Store, process, and retrieve
memories automatically

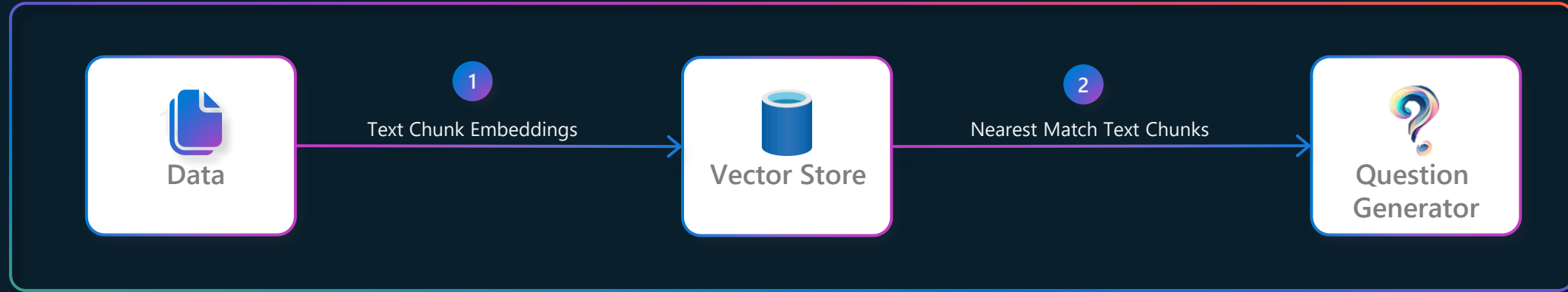
Facts, summaries, procedural,
episodic, & more



Why Knowledge Graph as a data model?

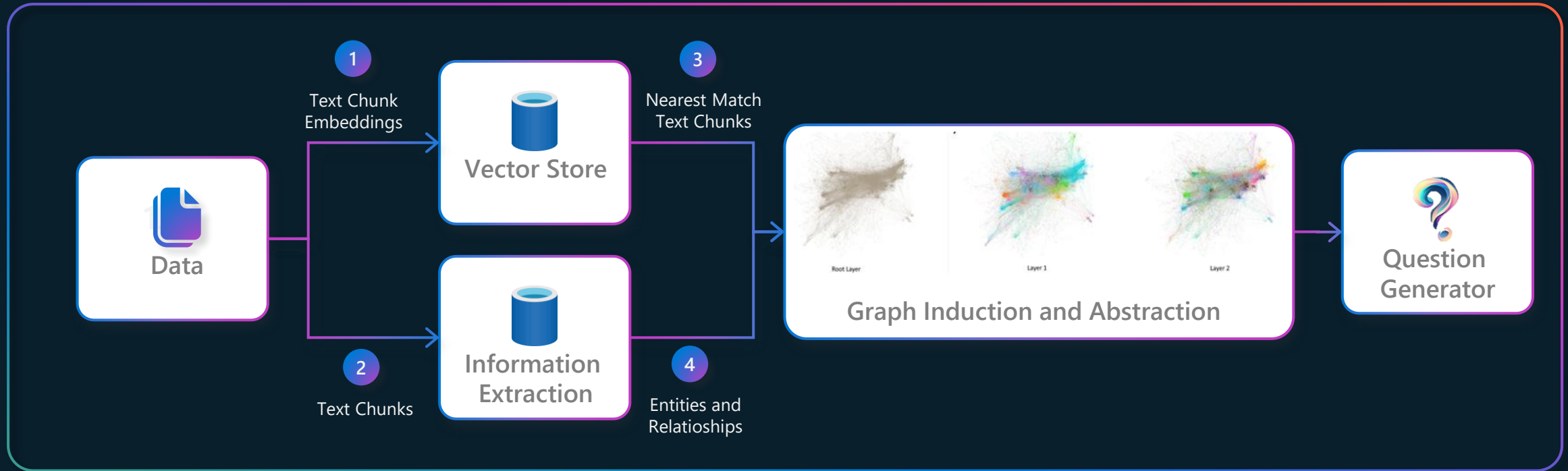
- Better at handling open-ended, nuanced and multi-hop queries
 - Provides more complete and contextually rich answers
- Provides grounding source and chain of thoughts
 - Enhances the traceability/audit of the responses
 - Supports hallucination checking
- Particularly effective for tasks that require reasoning across many documents/stores
 - Explores adjacent topics
 - Answers nuanced questions
- Bonus: saves AI costs by leveraging curated/prepped data, immediately usable by models for reasoning

Preprocessing Pipeline – Basic RAG



- ✓ Information lookup that is too big for context window
- ✗ Questions that require multi-hop reasoning
- ✗ Topics that have thin connections across many documents
- ✗ Abstractions and thematic retrievals

Preprocessing Pipeline – GraphRAG



- ✓ Information lookup that is too big for context window
- ✓ Questions that require multi-hop reasoning
- ✓ Topics that have thin connections across many documents
- ✓ Abstractions and thematic retrievals

Vector Search Results

Real-world example with lawsuits dataset

Recall = 10%

	Legal Case	Relevant
1	United Mutual Savings Bank v. Riebli (1960)	True
2	Wilkening v. State (1959)	False
3	Wilber Development Corp. v. Les Rowland Constr., Inc. (1974)	False
4	Washington Hydroculture, Inc. v. Payne (1981)	False
5	United Mutual Savings Bank v. Riebli (1960)	False
6	Strickland v. City of Seattle (1963)	False
7	King County v. Boeing Co. (1963)	False
8	DeHoney v. Gjarde (1925)	False
9	Evans v. Seattle (1935)	False
10	Arnold-Evans Co. v. Hardung (1925)	False

Semantic Ranker Results

Real-world example with lawsuits dataset

Recall = 20%

	Legal Case	Relevant
1	United Mutual Savings Bank v. Riebli (1960)	True
2	Javins v. First Nat'l Realty Corp (1970)	True
3	Geise v. Lee (1975)	False
4	McCutcheon v. United Homes Corp. (1971)	False
5	Reardon v. Shimelman (1925)	False
6	Woldson v. Woodhead (2005)	False
7	Donworth v. St. Paul Etc. R. Co. (1905)	False
8	Senske v. Washington Gas & Elec. Co. (1931)	False
9	Evans v. Seattle (1935)	False
10	Miller v. Vance Lumber Co.(1932)	False

GraphRAG Search Results

Real-world example with lawsuits dataset

Recall = 70%

Legal Case	Relevant
United Mutual Savings Bank v. Riebli (1960)	True
Javins v. First Nat'l Realty Corp (1970)	True
Foisy v. Wyman (1973)	True
Thomas v. Housing Authority (1967)	True
Jorgensen v. Massart (1963)	True
Martindale Clothing Co. v. Spokane & Eastern Trust Co. (1914)	True
Stuart v. Coldwell Banker (1987)	True
Papac v. City of Montesano (1956)	False
Finley v. City of Puyallup (1957)	False
Bach v. Sarich (1968)	False

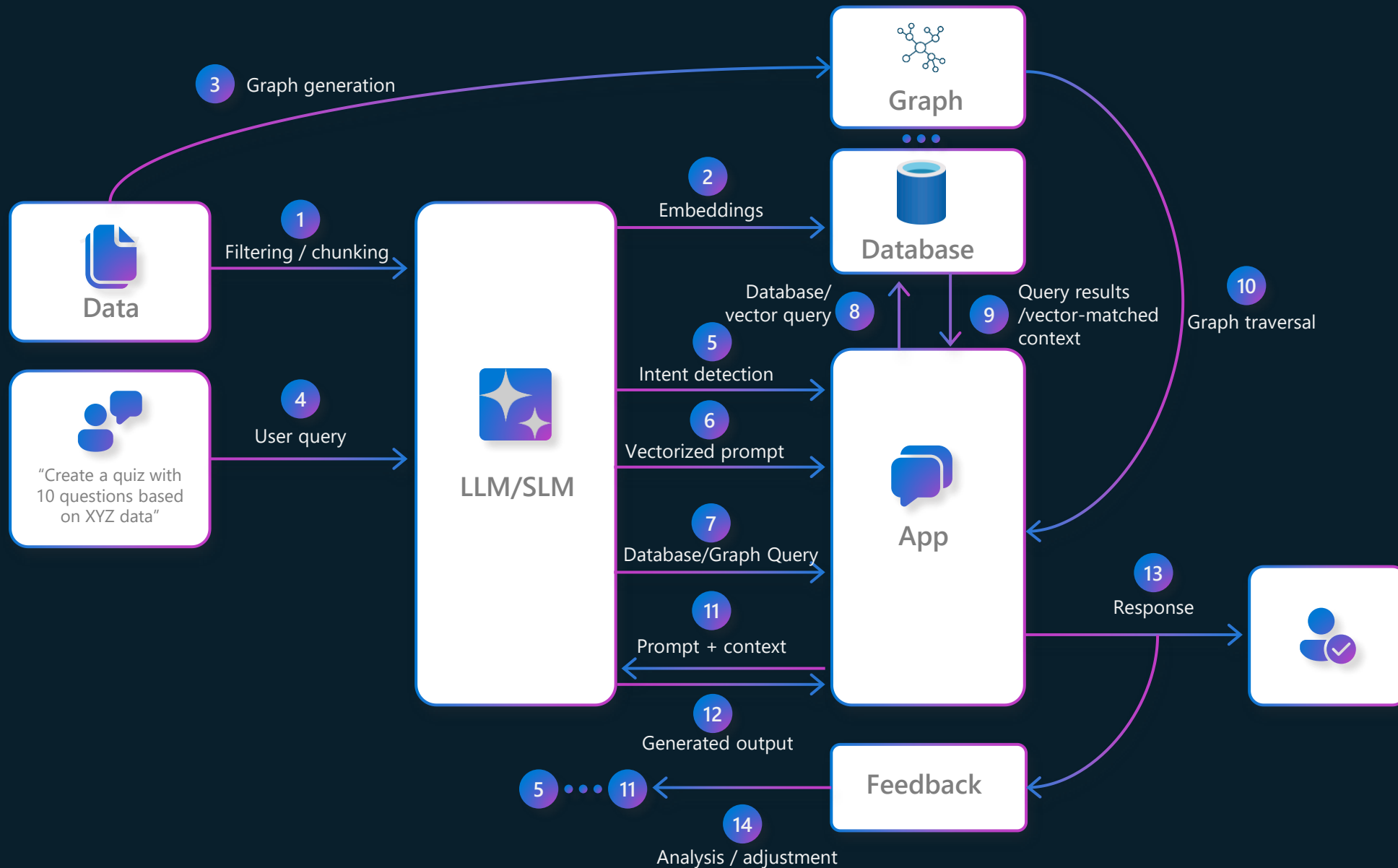
Graph generation options

- From structured data - data wrangling
 - Create triples (subject/predicate/object) from well-known structured data
 - [Optional] Create structured data out of unstructured (e.g. with Azure AI Document Intelligence) for comprehensive coverage
- From unstructured data - GraphRAG Indexing API
 - Use GraphRAG Query API for querying with NL
 - [Optional] Export graph to JSONL and then to TTL for import to triple store for faster/scalable traversal

Beyond graph

- What else do we need for fast and accurate context retrieval?
 - Graph IS NOT an appropriate data structure for some types of data (e.g. images, time series, transactions, etc.), so it's necessary but not enough by itself
 - Generating the graph for some datasets/data sources is not feasible
 - There always will be sources of data available for querying but not full export and conversion for various reasons (CRM, ERP, HRM, cache, data lakes, data warehouses, etc.)
- MULTIPLE SOURCES OF DATA MUST BE USED IN CONTEXT RETRIEVAL, not just vector or graph so virtualization of context retrieval across multiple sources is needed:
 - Preferably seamless to the user (developer retrieving the context)
 - Preferably leveraging the results from one source to retrieve better context from others
 - Graph can still be (and better be) used to connect them all through hyperlinks.

OmniRAG Pattern



OmniRAG Core Tenets

- **Omni-source with data virtualization**
 - Not limited to vector store, utilizing ALL data sources that can bring value to the context for AI
 - Use data wherever it is, in the original format, minimizing data movement and transformation
- **Knowledge graph**
 - Contains entities/relationships from existing data to make it readily available for AI to reason over (along with original data)
- **User intent detection**
 - Allows automatic routing of user's query to the right source, leverages AI
- **Runtime NL2Query conversion**
 - Convert user query to the source's query language using simple utterance analysis and/or AI
- **Session analytics**
 - Required to fine-tune golden dataset of questions + intent so intent detection improves. Could be used for semantic cache generation/curation

RAG Pattern Comparison: Data/Query Support

		Basic RAG	Community GraphRAG (MSR GraphRAG, LazyGraphRAG*, GraphRAG Gen 3*)	DiverseRAG**	OmniRAG (Basic RAG/GraphRAG on structured + Community GraphRAG on unstructured)
Unstructured Data	Information retrieval	✓	✓	✓	✓
	Multi-hop questions	✗	✓	✓	✓
	Open-ended questions	✗	✓	✗	✓
	Nuanced questions	✗	✓	✗	✓
Structured Data	Local search	✓		✓	✓
	Global search	✗		✓	✓
	Multi-hop queries	✗		✓	✓

* - not available publicly

** - available per request as open-source, no changes to service needed

RAG Pattern Comparison: Capabilities

	Basic RAG	GraphRAG	OmniRAG
Sources	Vector	Graph + Vector	Graph, Vector, Database, REST API, Files, etc.
Intent Detection	N/A	N/A	Yes
Semantic Model Generation	Generic (embeddings)	Generic (ontology + embeddings)	Custom (with code) or assisted by GraphRAG Zero
NL2Query	N/A	Yes (opaque)	Yes (transparent)
Global Search	No	Yes	Yes
Scalable	No	No	Yes (leverages native scaling of each source)
Context Awareness	Some	Deep	Very Deep
Direct hit search results	✓	✓	✓
Topically relevant connections	✗	✓	✓
Deep and sparse connections	✗	✓	✓
"Question behind the question"	✗	Partial	✓
Hallucination Risk	Low	Very Low	Very Low
Hallucination Checking	✗	✓	✓



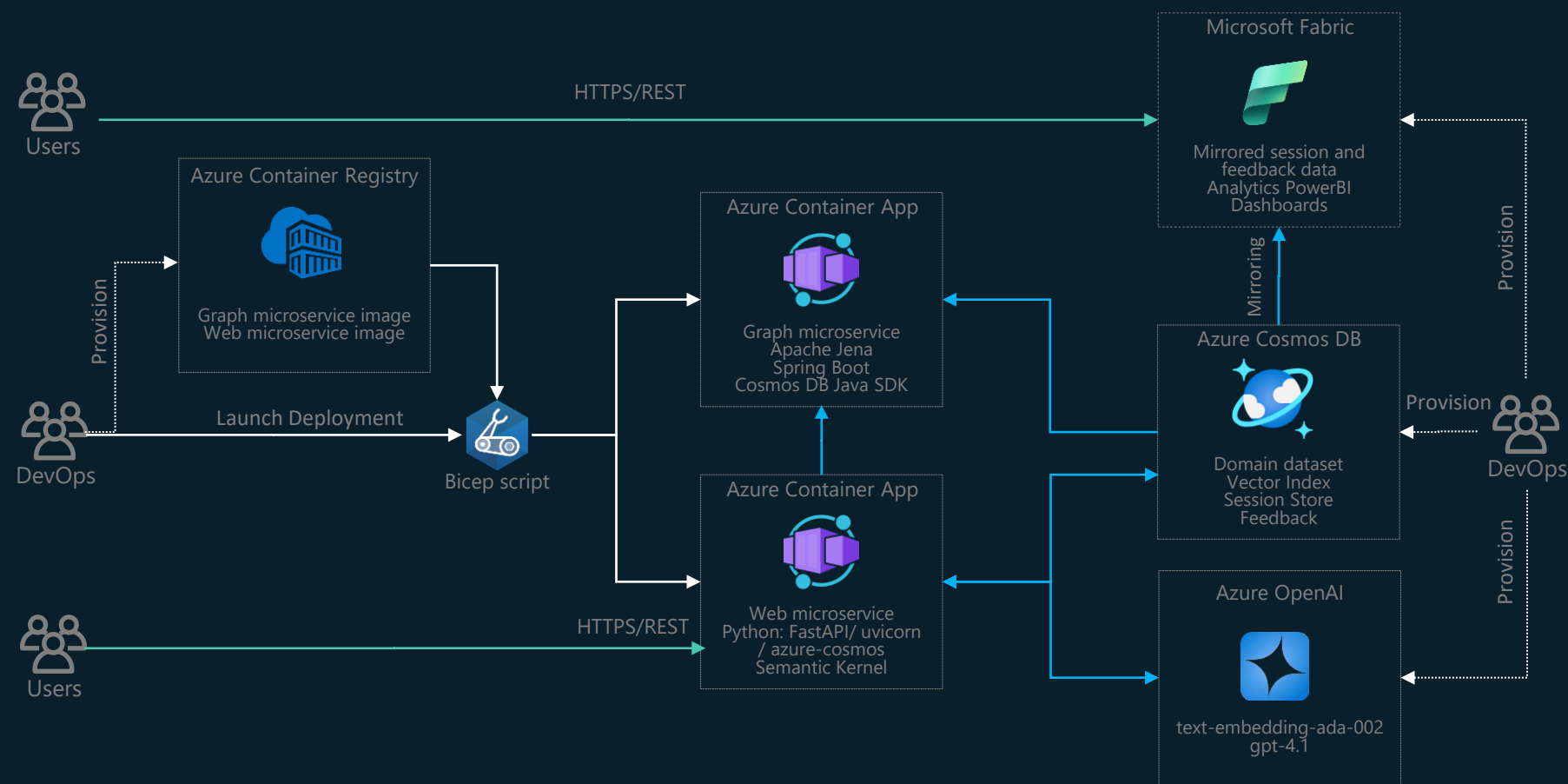
CosmosAI Graph as OmniRAG implementation

aka.ms/caig

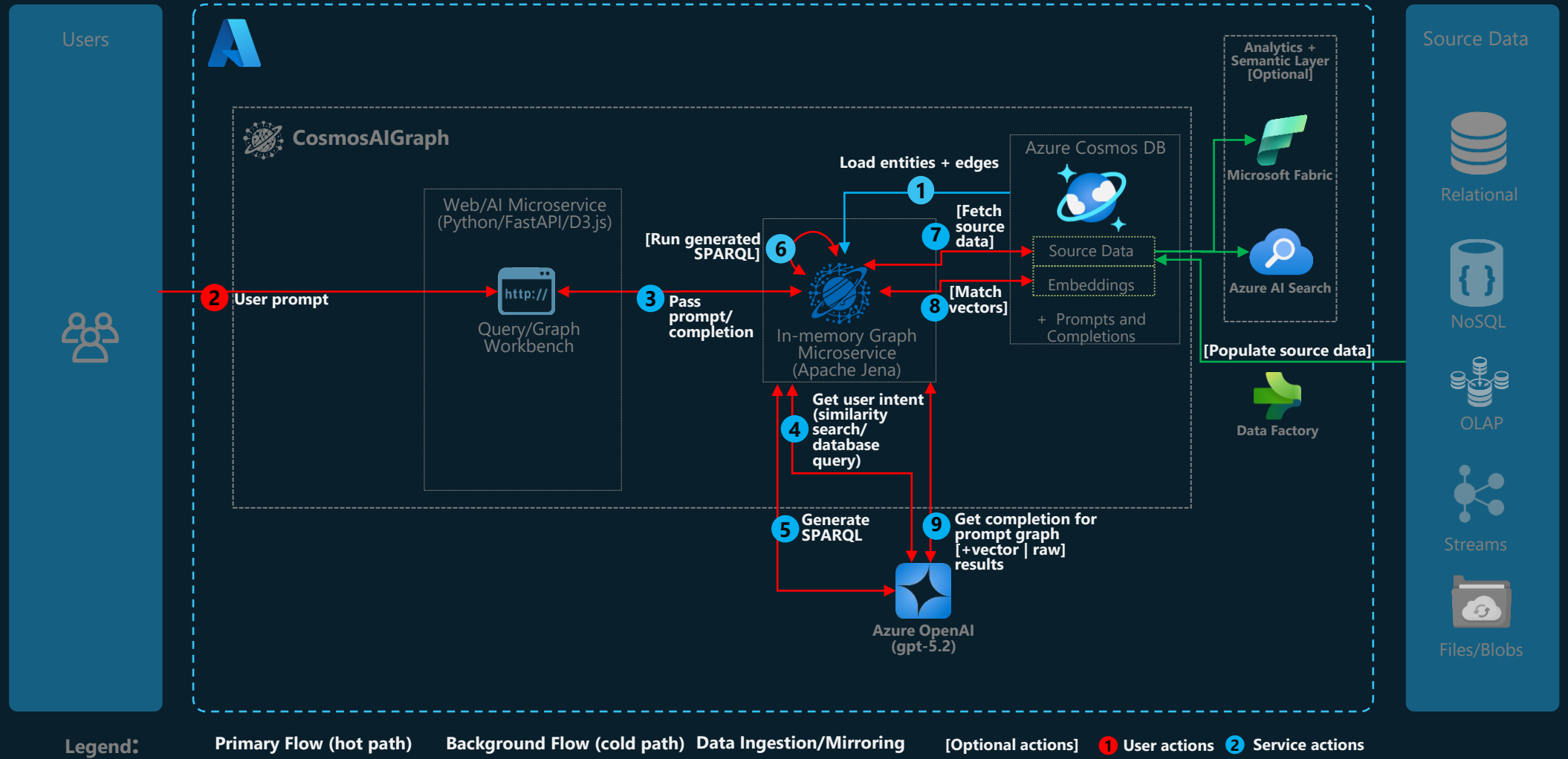
What is it providing?

- OmniRAG pattern reference implementation using knowledge graph/vector database/raw database
- Knowledge graph construction from structured data (Azure Cosmos DB, TTL file)
- User intent detection powered by built-in keyword scan and AI
- Scalable indexed RDF triple store database with in-memory model and persistent store for graph and domain data (Azure Cosmos DB)
- “Natural language to graph query” generator powered by AI with optional feedback loop

CosmosAIGraph Deployment Architecture



CosmosAIGraph Solution Architecture



CosmosAIGraph Summary



- It's a reusable reference design and implementation. **It's not a product**
- It is built on the following:
 - **Azure Cosmos DB for NoSQL** service, natively supporting vector search (DiskANN)
 - **RDF** technologies – triples, ontologies, SPARQL queries
 - **Apache Jena** – in-memory indexed RDF graph for fast performance and low costs
 - **Python 3** – UX, DAL and orchestration with fastapi, pydantic, azure-cosmos
 - **Microsoft Foundry** service with industry-leading LLMs/SLMs
 - **Microsoft Agent Framework** – for pluggable and extensible orchestration
- Designed as set of microservices
- Deployed to Azure Container Apps with Bicep

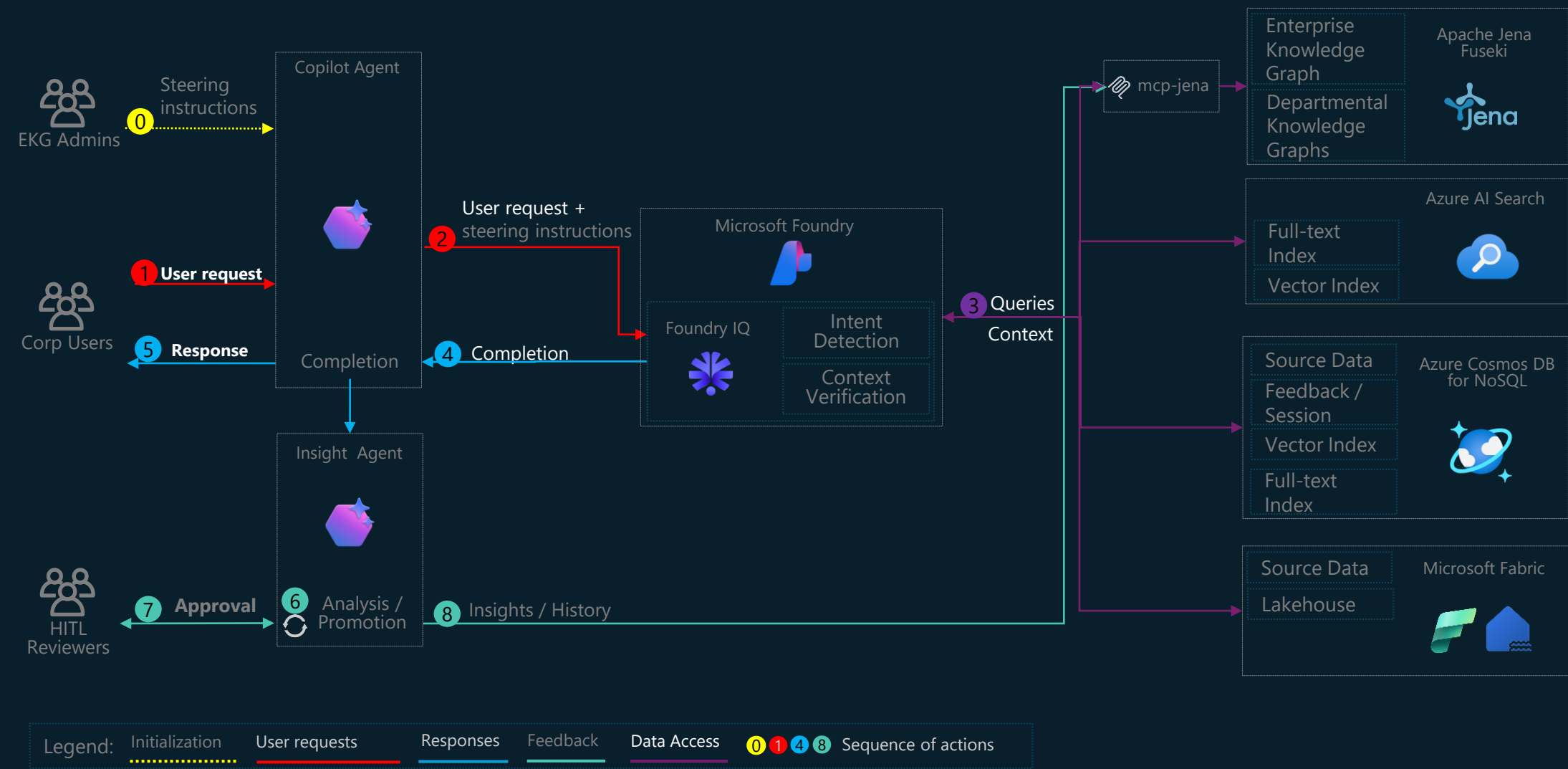
Graph/GraphRAG use cases and solutions

	Structured	Unstructured
Single Source	CosmosAIGraph (Cosmos DB + Apache Jena) Azure Database for PostgreSQL w/ Apache AGE Graph in Fabric	DiverseRAG (Cosmos DB + client reranker) GraphRAG Agent Explorer GraphRAG Gen 3 (binary distribution)
Multiple Sources	CosmosAIGraph [+ DiverseRAG (Cosmos DB + client reranker)] [+ custom connectors] Graph in Fabric	CosmosAIGraph [+ DiverseRAG (Cosmos DB + client reranker)] [+ GraphRAG Gen 3 (binary distribution)] [+ custom connectors]

Solution Customization

- **For real-time GraphRAG/DiverseRAG on unstructured (customer-generated) data:**
 - [Easy] Adopt latest GraphRAG Gen 3, and [optionally] support streaming updates to the underlying data
 - [Medium] Export GraphRAG Gen 3 Indexing API output and load it into in-memory graph database and work with its query API instead
- **For broad support of various customer data sources:**
 - Develop agentic framework (see next slide) to support pluggable agents supporting various data sources so they can all participate in providing context in OmniRAG
- **For flexible/configurable orchestration** (Agentic RAG):
 - Develop rule designer, supporting multiple workflow primitives
 - Implement best practice workflows/rules by default for easy start.

Enterprise Knowledge Graph Architecture



Call to Action!

- ✓ Go to public repo aka.ms/CAIG and adopt to your structured dataset. Deploy it to your Azure subscription or run locally
- ✓ Go to public repo aka.ms/AgenticRetrieval and adopt to your unstructured dataset and test locally
- ✓ Explore <https://learn.microsoft.com/en-us/azure/cosmos-db/gen-ai/agent-memory-toolkit> for agent memory toolkit and <https://learn.microsoft.com/en-us/azure/cosmos-db/gen-ai/semantic-reranker> for semantic reranker
- ✓ Give us your feedback on DiverseRAG, OmniRAG and CosmosAIGraph!